

Webapp per la gestione di aree cani

Pietro Ciani



Ingegneria del software

Professor Enrico Vicario

Ingegneria Informatica

Anno Accademico 2024/2025

Indice

1	Introduzione	2
1.1	Tecnologie impiegate	2
2	Progettazione	3
2.1	Use Case Diagram	3
2.2	Use Case Templates	3
2.2.1	Casi d'uso principali di un utente	3
2.3	Mockups	5
2.3.1	Navbar	5
2.3.2	Park's Reservation form	5
2.3.3	My Reservation's Reservation form	5
2.3.4	Homepage	6
2.3.5	Logged user Homepage	6
2.3.6	List of Parks	7
2.4	Class Diagram	7
2.5	DAO Pattern	9
2.6	Entity Relationship Diagram	9
3	Implementazioni delle classi	10
3.1	Model	11
3.1.1	Park	11
3.1.2	Reservation	11
3.1.3	User	11
3.2	Controller	11
3.2.1	HomeController	11
3.2.2	ParkController	12
3.2.3	ReservationController	12
3.2.4	SignupController	13
3.3	Service	13
3.3.1	ReservationService	13
3.4	DAO	14
3.4.1	Spring Data JPA	14
3.5	Security	14
3.5.1	CustomUserDetailsService	15
3.5.2	WebSecurityConfig	15
4	GUI	16
5	Test (JUnit)	18
5.1	Model Test	19
5.2	Controller Test	21
6	Note sulla connessione al Database	24
6.1	DBMS: PostgreSQL	24

1 Introduzione

BarkPark: Innovazione Digitale per il Benessere Canino

Nel panorama urbano contemporaneo, la gestione degli spazi dedicati ai cani è affidata agli utenti stessi. Questa pessima organizzazione di tali spazi dà luogo a numerosi problemi e rende un'uscita con il proprio amico a quattro zampe una scommessa.

BarkPark nasce come soluzione tecnologica innovativa per rispondere a queste esigenze, offrendo una piattaforma digitale che semplifica la fruizione e la gestione delle aree cani. Attraverso un'interfaccia intuitiva BarkPark mira a migliorare la qualità della vita dei cani e dei loro proprietari.

1.1 Tecnologie impiegate

Per la realizzazione della web app si è scelto un Spring Boot, un framework Java basato sull'architettura MVC (Model View Controller).

Il framework include pacchetti opzionali come Spring Security, per la gestione degli accessi e dei diritti dei singoli utenti e Spring Data JPA per semplificare ed automatizzare l'interazione con il database.

Si è utilizzato PostgreSQL come Database Management System insieme a pgAdmin 4 per la gestione e l'hosting del database in locale ma questo non esclude che il database possa in futuro essere spostato su una macchina esterna. In questo caso abbiamo creato due database, uno per l'app, contenente i dati degli utenti veri e propri e uno per la fase di test, per evitare che le query eseguite in fase di test vadano a corrompere i dati già salvati sul database dell'app.

Si è utilizzato anche Lombok, una libreria per ridurre il boilerplate nel codice Java; essa infatti genera automaticamente metodi come getter e setter utilizzando annotazioni come @Getter e @Setter.

Per la fase di test è stato utilizzato JUnit, una libreria per unit testing per assicurare la correttezza del codice.

La parte frontend del progetto è stata realizzata con Bootstrap, un framework CSS per la creazione di interfacce responsive e moderne e attraverso l'uso di Thymeleaf, un template engine utilizzato per creare pagine HTML mantenendo una coerenza grafica in tutte le schede e riducendo il volume di codice ripetuto.

Nella parte di sviluppo del codice sono stati utilizzati tool come GitHub Copilot che forniscono completamento automatico del codice attraverso l'uso di intelligenza artificiale generativa. Tutto il codice generato dai suddetti strumenti è stato oggetto di revisione ed adattamento per verificarne il corretto comportamento.

2 Progettazione

Possiamo distinguere 5 pacchetti principali:

- Model contiene le classi che rappresentano le entità principali dell'applicativo. Definisce le tabelle del database e le loro relazioni.
- Controller gestisce le richieste HTTP, mappa gli URL ai metodi corrispondenti e fornisce le viste appropriate.
- Service contiene la logica applicativa e funge da intermediario tra il livello Controller e il livello DAO. E' il livello che implementa la logica più complessa come la gestione delle prenotazioni o la verifica delle regole di business.
- DAO (Data Access Object) fornisce l'accesso al database e gestisce operazioni CRUD sulle entità. Utilizza Spring Data JPA per interagire con il database tramite metodi predefiniti.
- Security gestisce l'autenticazione e l'autorizzazione degli utenti, configura la sicurezza dell'applicazione tramite WebSecurityConfig per proteggere le risorse e definisce meccanismi per login, logout e registrazione.

I pacchetti e le loro interazioni sono schematizzati nello schema che segue (Fig. 1).

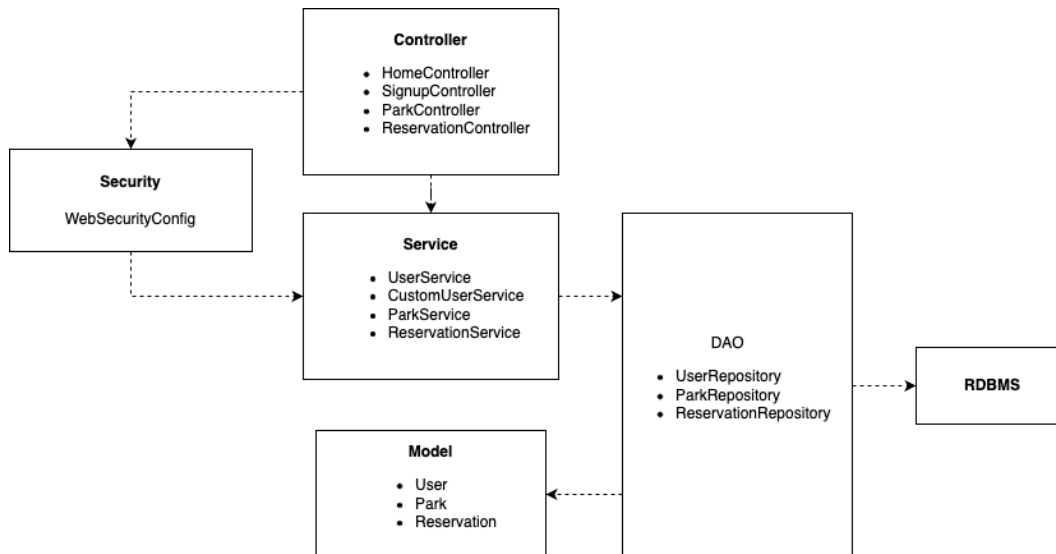


Figura 1: Package Diagram

2.1 Use Case Diagram

La piattaforma è popolata da utenti registrati, rappresentati dall'entità User, che possono creare prenotazioni del parco selezionato, indicato dall'entità Park. Le prenotazioni, Reservation, possono essere create, visualizzate ed eliminate dall'area personale.

Lo schema dei casi d'uso è realizzato secondo lo standard UML mediante StarUML

2.2 Use Case Templates

Di seguito sono illustrati i casi d'uso principali nel dettaglio.

2.2.1 Casi d'uso principali di un utente

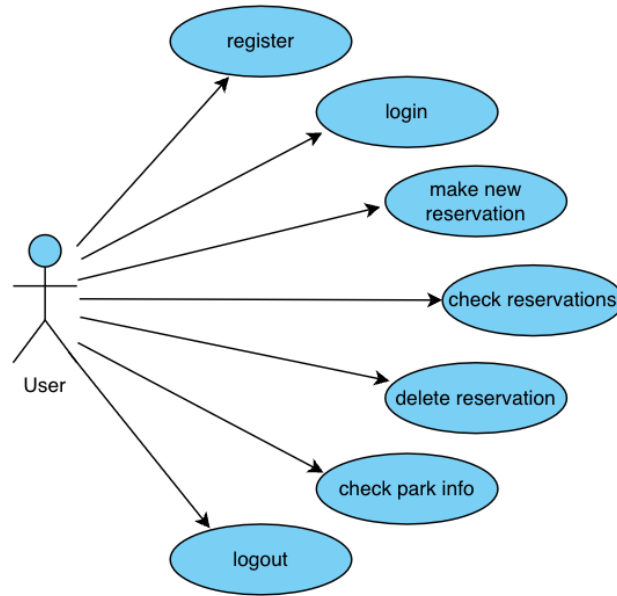


Figura 2: Use Case Diagram

UC-1	Utente crea una Prenotazione
Descrizione	L'utente verifica gli orari disponibili e crea una prenotazione di conseguenza.
Livello	User goal
Azioni	1. L'utente clicca nella sezione "Parks" della navbar superiore (Fig. 3) o dalla card dedicata (Fig. 7). 2. Il client riceve dal database una lista di parchi e la mostra all'utente (Fig. 8). 3. L'utente sceglie il parco cliccando sulla specifica card dedicata. 4. Viene mostrata la pagina dedicata al parco con informazioni come la posizione, l'orario di apertura e le prenotazioni già effettuate da altri utenti nascondendone i nomi. 5. L'utente riempie i campi sul lato destro specificando la data, l'ora e la durata della prenotazione e clicca su Reserve (Fig. 4).

Tabella 1: Creazione di una Prenotazione da parte di un Utente registrato

UC-2	Utente gestisce le proprie prenotazioni
Descrizione	L'utente visualizza, crea o elimina le proprie prenotazioni
Livello	User goal
Azioni	1. Dalla homepage l'utente seleziona "My Reservations" dalla navbar (Fig. 3) o dalla card dedicata (Fig. 7). 2. Viene quindi mostrata una tabella con tutte le prossime prenotazioni effettuate dall'utente. 3. E' possibile a questo punto eliminare una prenotazione non più utile o effettuare nuove prenotazioni dal form mostrato sulla destra (Fig. 5).

Tabella 2: Gestione delle prenotazioni future dell'Utente

2.3 Mockups

In seguito sono stati riportati alcuni mockups, come screenshot della GUI realizzata per il progetto.

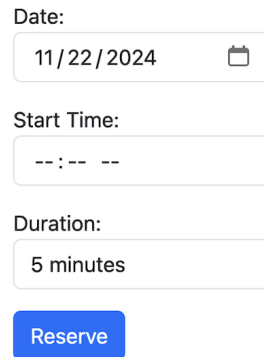
2.3.1 Navbar



BarkPark Home Parks My Reservations

Figura 3: Screenshot della navbar (UC-1.1, UC-2.1)

2.3.2 Park's Reservation form



Date:
11/22/2024

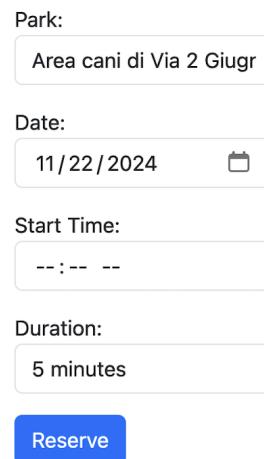
Start Time:
--:-- --

Duration:
5 minutes

Reserve

Figura 4: Screenshot del form per la prenotazione di un parco (UC-1.5)

2.3.3 My Reservation's Reservation form



Park:
Area cani di Via 2 Giugr

Date:
11/22/2024

Start Time:
--:-- --

Duration:
5 minutes

Reserve

Figura 5: Screenshot del form per la prenotazione di un parco dall'area personale (UC-2.3)

2.3.4 Homepage

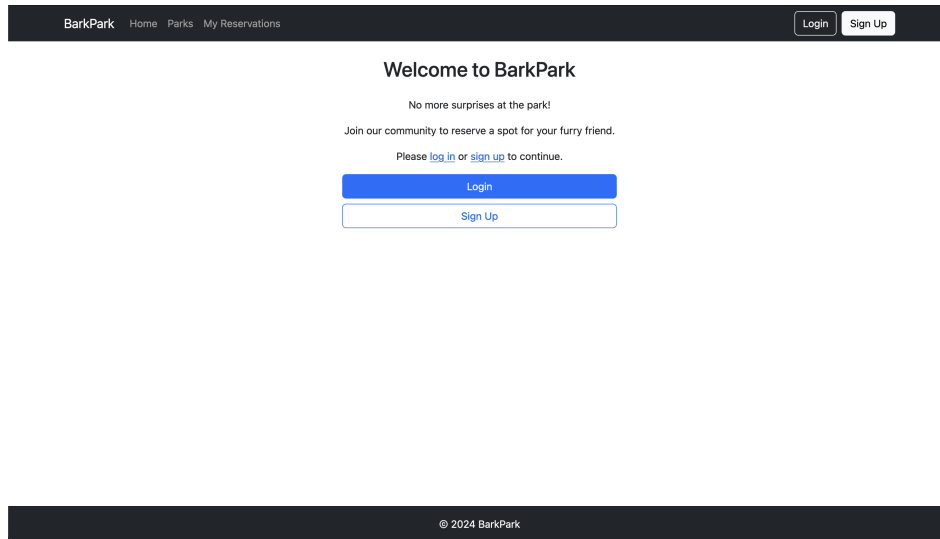


Figura 6: Screenshot della homepage di BarkPark

2.3.5 Logged user Homepage

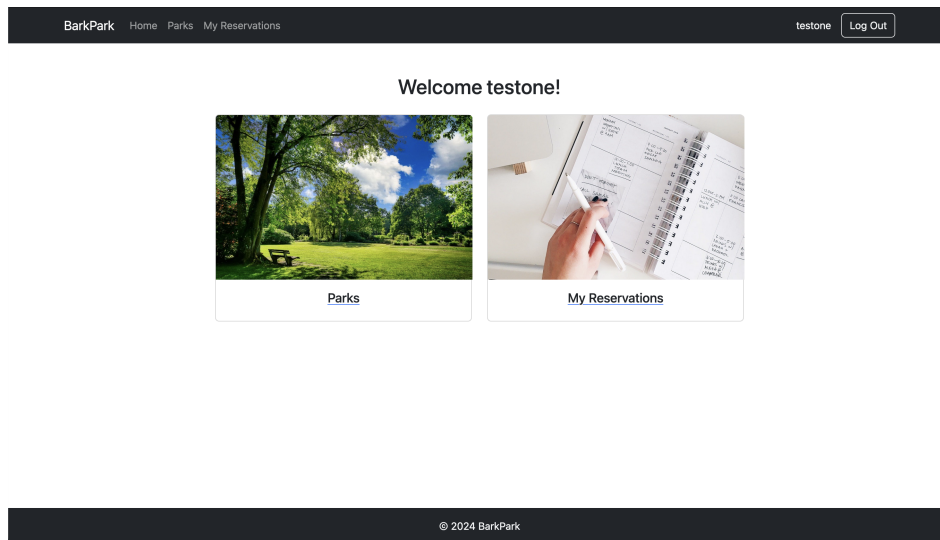


Figura 7: Screenshot della homepage per l'utente loggato

2.3.6 List of Parks

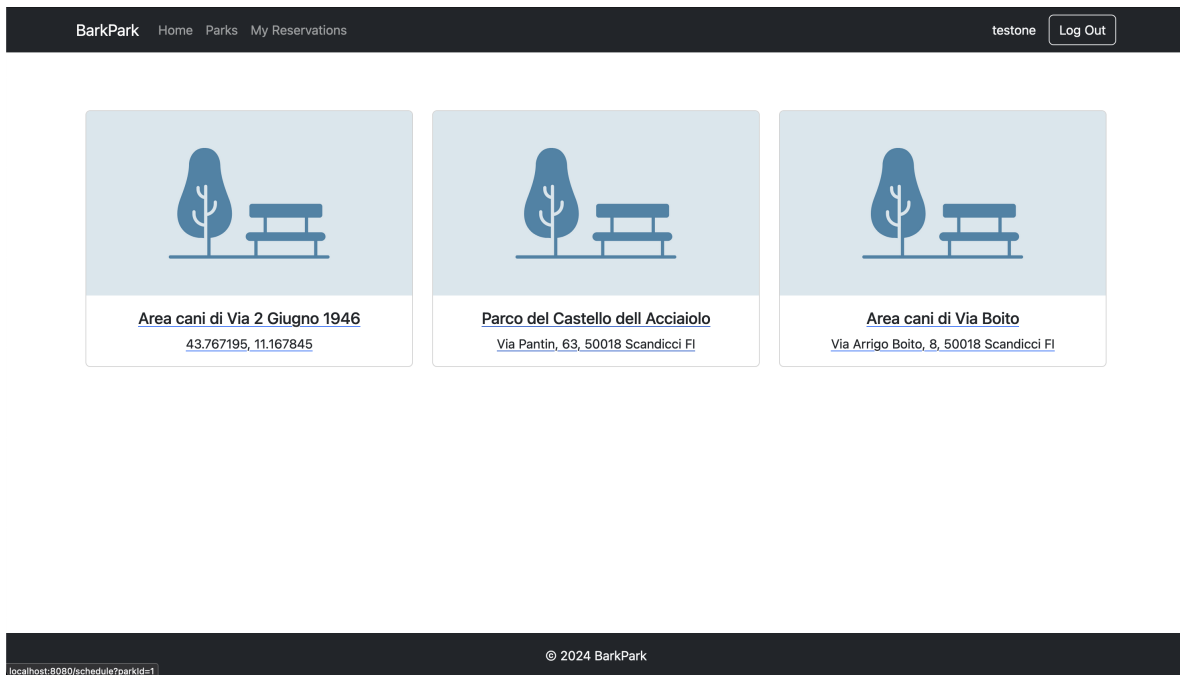


Figura 8: Screenshot della lista di tutti i parchi prenotabili dall'utente

2.4 Class Diagram

Di seguito è riportato il diagramma delle classi del software (Figura 2) e come queste sono legate e interagiscono tra loro.

I package contenuti all'interno del software sono i seguenti:

- MVC - Model View Controller: rappresenta il core del progetto. Contiene le principali classi che permettono di far funzionare l'applicazione.
- DAO: contiene le classi che permettono l'interazione col database.
- Security: rappresenta l'insieme di classi che si occupano della sicurezza degli account degli utenti usando password criptate e nascondendo agli utenti non loggati informazioni personali sensibili degli utenti registrati.

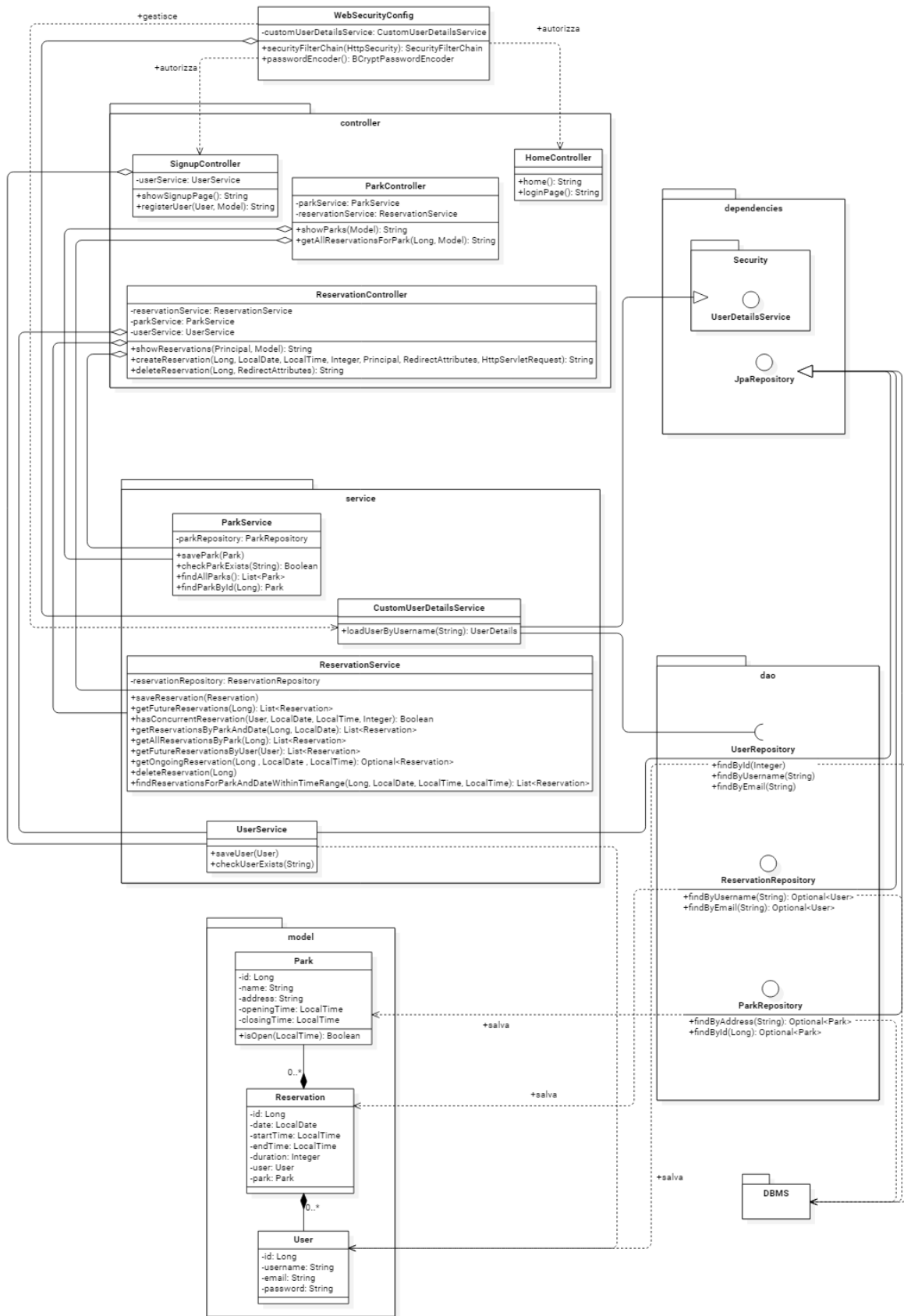


Figura 9: Diagramma UML

2.5 DAO Pattern

Per fare in modo che il client non possa interagire direttamente con il Database, è necessaria l'implementazione del Design Pattern DAO.

DAO (Data Access Object) è un design pattern architetturale che permette di mantenere la persistenza dei dati che sono registrati in un database. I dati rimangono in oltre coerenti durante l'intera esecuzione del programma e continuano ad esserlo anche dopo la terminazione. Il seguente pattern consente inoltre di separare la logica di accesso ai dati dal resto dell'applicazione. È progettato per isolare il codice che gestisce l'accesso e la manipolazione dei dati lato server.

Le singole repository di questo pacchetto sono in realtà interfacce che si limitano a dichiarare i metodi che poi verranno implementati automaticamente da Spring Data JPA. Quest'ultimo ecosistema fornisce metodi predefiniti per eseguire operazioni CRUD (Create, Read, Update, Delete) su entità gestite da un database relazionale attraverso JPA (Java Persistence API).

Le interfacce che estendono JpaRepository sono ParkRepository, ReservationRepository e UserRepository.

2.6 Entity Relationship Diagram

La seguente immagine rappresenta il diagramma Entity-Relationship del database realizzato per il progetto.

La cardinalità di Park e User con Reservation è di 1:n.

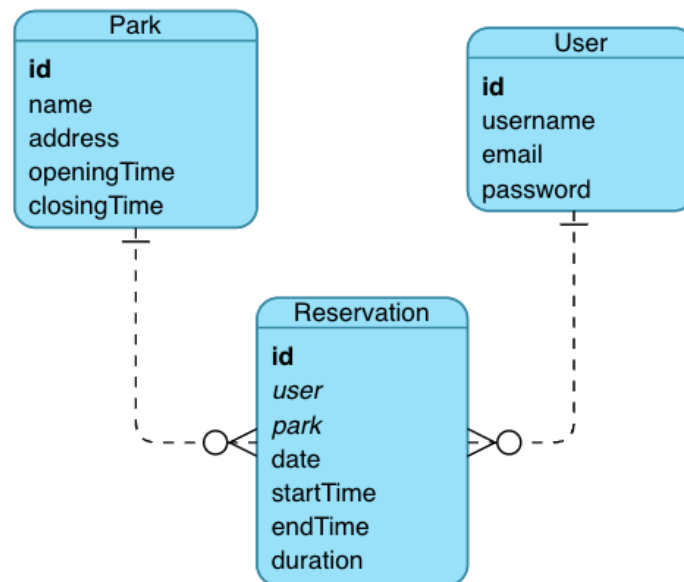


Figura 10: Diagramma ER

3 Implementazioni delle classi

Per questioni di chiarezza, il progetto è stato diviso in packages dove DAO viene usato per le interazioni con il database, Controller per gestire la mappatura delle pagine web, Service per l'implementazione delle varie operazioni, Model per definire le varie entità che verranno istanziate nel database e Security per implementare tutti i controlli di sicurezza forniti da SpringBoot Security. Il codice sorgente è organizzato nel seguente modo:

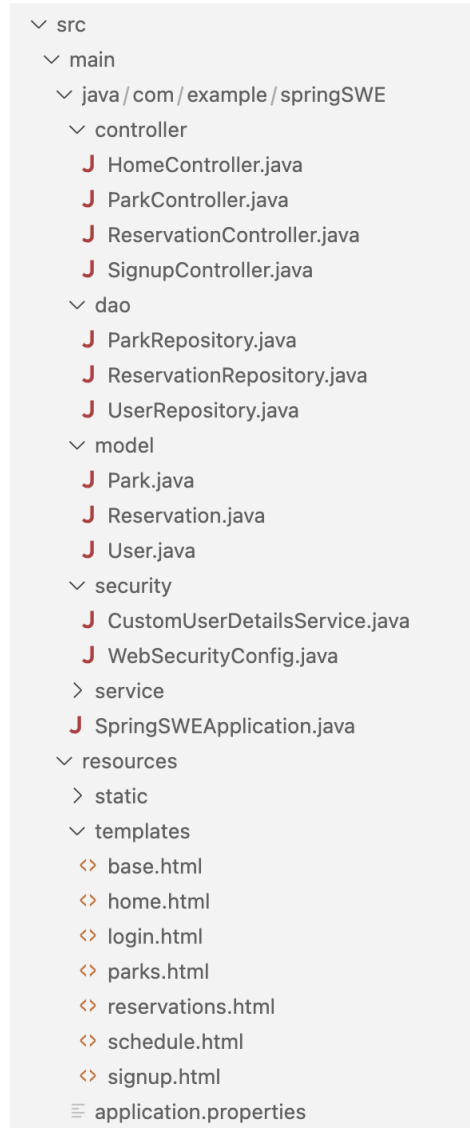


Figura 11: Suddivisione dei package all'interno dell'IDE

3.1 Model

3.1.1 Park

Park è la classe che rappresenta i parchi che saranno prenotati dagli utenti. La classe è composta dai seguenti attributi:

- id: l'identificativo univoco del parco
- name: indica il nome del parco
- address: è l'indirizzo del parco.
- openingTime: orario di apertura
- closingTime: orario di chiusura

3.1.2 Reservation

La classe Reservation rappresenta l'implementazione della prenotazione dell'utente del parco scelto all'orario selezionato. Gli attributi della classe sono:

- id: identificativo univoco della prenotazione
- date: indica in cui il parco è stato riservato
- startTime: indica l'orario di inizio della prenotazione
- endTime: indica l'orario al quale la prenotazione termina
- duration: indica la durata della prenotazione
- user: rappresenta l'utente che ha effettuato la prenotazione
- park: indica il parco prenotato

3.1.3 User

Rappresenta l'utente che ha effettuato la registrazione. I suoi attributi sono:

- id: identificativo univoco del cliente.
- username
- email
- password

3.2 Controller

Il package Controller permette di mappare le operazioni definite nel resto del codice ad indirizzi da cui possano essere raggiunte. Alcuni dei metodi di questo package sono molto semplici, con l'unico scopo di ritornare la vista corrispondente per una determinata pagina. Questi metodi, come `home()`, sono utilizzati per associare un indirizzo alla vista che deve essere mostrata. Di seguito viene mostrato un semplice esempio di controller ed in seguito solo i controller che abbiano una rilevanza logica nella manipolazione dei dati.

3.2.1 HomeController

La classe HomeController permette di visualizzare la homepage e la schermata di login. `home()` è un semplice caso di metodo il cui unico scopo è ritornare la homepage.

```
@GetMapping("/")
public String home() {
    return "home";
}
```

3.2.2 ParkController

La classe ParkController consente di visualizzare la lista di parchi prenotabili e le prenotazioni già effettuate nel parco selezionato attraverso l'interazione con il Service layer.

Di seguito si ha un metodo in cui viene ritornata la vista e i relativi dati, forniti dall'apposito Service, da mostrare in essa.

```
@GetMapping("/parks")
public String showParks(Model model) {
    List<Park> parks = parkService.findAllParks();
    model.addAttribute("parks", parks);
    return "parks";
}
```

3.2.3 ReservationController

ReservationController permette di visualizzare la lista di prenotazioni, creare o eliminare prenotazioni effettuate dall'utente loggato.

In questa classe si ha un livello di interazione con i dati ancora maggiore rispetto ai precedenti Controller e il rispettivo Service ricopre un ruolo ancora più importante.

Il metodo `createReservations()` gestisce la richiesta di creazione di una nuova prenotazione. Prima di salvare la nuova Reservation, il blocco try si occupa di verificare la correttezza dei dati inseriti e di impedire la creazione di eventuali prenotazioni sbagliate.

```
@PostMapping("/reservation/create")
public String createReservation(@RequestParam("parkId") Long parkId,
                                @RequestParam("date") @DateTimeFormat(pattern = "yyyy-MM-dd")
                                LocalDate date,
                                @RequestParam("startTime") @DateTimeFormat(pattern = "HH:mm")
                                LocalTime startTime,
                                @RequestParam("duration") int durationInMinutes,
                                Principal principal, RedirectAttributes redirectAttributes,
                                HttpServletRequest request) {

    String referer = request.getHeader("Referer");
    try {
        User user = userService.getUserByUsername(principal.getName());
        Park park = parkService.findParkById(parkId);
        LocalDateTime reservationDateTime = LocalDateTime.of(date, startTime);
        LocalTime endTime = startTime.plusMinutes(durationInMinutes);

        // Validate
        String validationError = validateReservation(parkId, date, startTime, endTime,
        park, user, reservationDateTime, durationInMinutes);
        if (validationError != null) {
            redirectAttributes.addFlashAttribute("error", validationError);
            return "redirect:" + (referer != null ? referer : "/");
        }

        Reservation reservation = new Reservation(date, startTime, durationInMinutes,
        user, park);
        reservationService.saveReservation(reservation);
        redirectAttributes.addFlashAttribute("success", "Reservation successfully created!");
        return "redirect:/reservations";

    } catch (Exception e) {
        redirectAttributes.addFlashAttribute("error", "Could not create reservation: "
        + e.getMessage());
    }
}
```

```

        return "redirect:" + (referer != null ? referer : "/");
    }
}

```

3.2.4 SignupController

La classe SignupController fornisce i metodi per creare un account. I principali metodi implementati nella classe sono i seguenti:

Una volta compilato il form per la registrazione nella pagina di sign up, i dati vengono passati ad UserService che si occuperà, attraverso il metodo `saveUser()` di salvare l'utente nell'apposita tabella del database.

```

@PostMapping
public String registerUser(@ModelAttribute User user, Model model) {
    if (userService.checkUserExists(user.getUsername(), user.getEmail())) {
        model.addAttribute("errorMessage", "Username or Email already exists");
        return "signup";
    }
    userService.saveUser(user);
    return "redirect:/login";
}

```

3.3 Service

Le classi Service ricoprono un ruolo fondamentale nell'architettura dell'applicazione, fungendo da livello intermedio tra le repository dei dati e i controller. A queste classi è deputata l'implementazione della logica di business dell'applicazione, rappresentando lo strato in cui vengono definite le operazioni che manipolano e lavorano i dati.

In questo package figurano ParkService, UserService e ReservationService. Approfondiremo solo l'ultima in quanto le prime due fungono da semplici interfacce tra le rispettive repository e i controller.

3.3.1 ReservationService

Contiene la logica secondo cui le prenotazioni vengono scelte e mostrate dai vari controller. `getFutureReservationsByUser()` ritorna le prenotazioni future dell'utente loggato ordinandole dalla più vicina alla più lontana nel tempo.

```

public List<Reservation> getFutureReservationsByUser(User user) {
    LocalDate today = LocalDate.now();
    LocalTime now = LocalTime.now();

    List<Reservation> reservations = reservationRepository.findById(user.getId())
        .stream()
        .filter(reservation ->
            reservation.getDate().isAfter(today) ||
            (reservation.getDate().isEqual(today) && reservation.getStartTime().isAfter(now))
        )
        .sorted(Comparator.comparing(Reservation::getDate).thenComparing(Reservation::getStartTime))
        .collect(Collectors.toList());
    return reservations;
}

```

`getOngoingReservation()` è un semplice metodo che ritorna un eventuale prenotazione in corso da parte dell'utente loggato.

```

public Optional<Reservation> getOngoingReservation(Long parkID, LocalDate date,
                                                    LocalTime startTime) {
    List<Reservation> reservations = reservationRepository

```

```

        .findOngoingReservationByParkDateTime(parkID, date, startTime);
    return reservations.isEmpty() ? Optional.empty() : Optional.of(reservations.get(0));
}

```

Nel metodo `hasConcurrentReservation()` ad esempio si verifica che l'utente che sta effettuando la prenotazione non abbia già un'altra Reservation al medesimo orario, in un altro o in quello stesso parco.

```

public boolean hasConcurrentReservation(User user, LocalDate date,
                                       LocalTime startTime, int durationInMinutes) {
    List<Reservation> userReservations = reservationRepository.findByUserAndDate(user, date);
    for (Reservation existingReservation : userReservations) {
        LocalTime existingStartTime = existingReservation.getStartTime();
        LocalTime existingEndTime = existingReservation.getEndTime();
        LocalTime newEndTime = startTime.plusMinutes(durationInMinutes);
        if ((startTime.isBefore(existingEndTime) && newEndTime.isAfter(existingStartTime)) ||
            startTime.equals(existingStartTime)) {
            return true;
        }
    }
    return false;
}

```

3.4 DAO

Il package DAO, Data Access Object, rende possibile l'interazione del programma con il database.

3.4.1 Spring Data JPA

Nel package DAO sono contenute le interfacce derivate da Spring Data JPA che permettono di separare il business layer, cioè l'applicazione, dal database. Spring Data JPA è un meccanismo che semplifica significativamente l'accesso ai dati nei database Java, introducendo un'innovativa capacità di generazione automatica delle query basata sui nomi dei metodi. Quando si definisce un'interfaccia repository estesa da `JpaRepository`, Spring Data JPA è in grado di interpretare il nome del metodo e trasformarlo automaticamente in una query specifica per il database. Questo avviene seguendo precise convenzioni di denominazione descritte nella documentazione Spring.

Di seguito alcuni esempi di metodi di `ReservationRepository`:

- `findByParkId` ritorna una lista di Reservation contenenti l'id del parco fornito
- `findByUserAndDate` ritorna invece tutte le prenotazioni effettuate dall'utente e nella data fornita.
- `findByParkIdAndDateAndStartTimeAfter` introduce un ulteriore livello di complicazione applicando una condizione sulla data.

```

List<Reservation> findByParkId(Long parkId)

List<Reservation> findByUserAndDate(User user, LocalDate date)

List<Reservation> findByParkIdAndDateAndStartTimeAfter(Long parkId,
                                                       LocalDate date, LocalTime startTime)

```

La stessa tecnica di costruzione e traduzione implicita delle query attraverso Spring Data JPA è stata applicata anche a `ParkRepository` e `UserRepository`.

3.5 Security

Questo package contiene tutti i metodi di cui SpringBoot Security necessita per permettere l'autenticazione degli utenti e limitare l'accesso ai dati altrui.

3.5.1 CustomUserService

Questa classe ha lo scopo di mettere a disposizione funzioni per la gestione delle autorizzazioni e dell'autenticazione dell'utente. In particolare loadUserByUsername è necessario al sistema di sicurezza di SpringBoot per l'autenticazione ed il caricamento dell'utente identificato dal proprio username univoco.

3.5.2 WebSecurityConfig

Questa classe ha lo scopo di gestire effettivamente i controlli di sicurezza della web app attraverso il metodo securityFilterChain.

securityFilterChain si occupa di filtrare le richieste di accesso alle varie parti del sito verificando che chi stia provando ad accedere abbia le giuste autorizzazioni per farlo.

```
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .userDetailsService(userDetailsService)
        .authorizeHttpRequests((requests) -> requests
            .requestMatchers("/", "/home", "/signup").permitAll()
            .anyRequest().authenticated()
        )
        .formLogin((form) -> form
            .loginPage("/login")
            .defaultSuccessUrl("/")
            .permitAll()
        )
        .logout(LogoutConfigurer::permitAll);

    return http.build();
}
```


4 GUI

La GUI del progetto è stata realizzata in HTML con l'ausilio di Bootstrap per permetterne una rapida realizzazione. Ho inoltre scelto di utilizzare il template engine Thymeleaf per evitare di ripetere codice già scritto e mantenere una coerenza grafica in tutte le pagine del progetto. L'interfaccia è composta principalmente dalle seguenti pagine web:

- base: è la pagina base usata come template per tutte le altre pagine, contiene elementi essenziali come il nome del sito, navbar e footer. La pagina non è raggiungibile dagli utenti in quanto non mappata da nessun controller.
- home: in questa sezione vengono riportate le funzioni principali implementate come card e i bottoni che reindirizzano ai form di registrazione e di login.

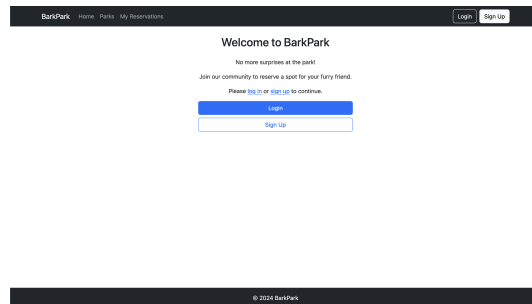


Figura 12: home.html

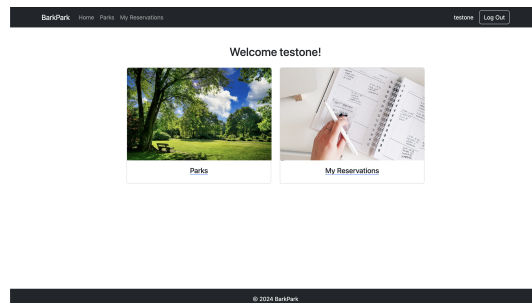


Figura 13: home.html per utente loggato

- signup: è la pagina che contiene il form che permette ai nuovi utenti di registrarsi attraverso l'uso di username ed email univoci associandovi una password.

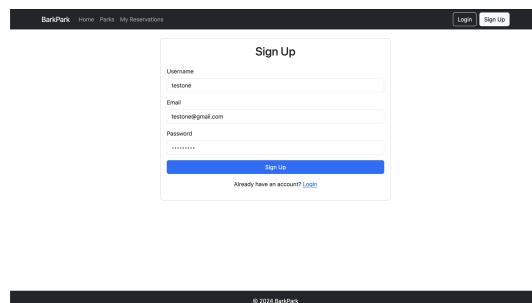


Figura 14: signup.html

- login: è la pagina che contiene il form che permette il login agli utenti già registrati.

Figura 15: login.html

Figura 16: parks.html

- parks: è la pagina in cui vengono mostrati tutti i parchi prenotabili dall'utente
- schedule: è la pagina in cui l'utente può vedere gli orari di apertura del parco e la bacheca contenente le prenotazioni già effettuate dall'utente stesso o da altri utenti (mostrati come anonimi ovviamente)

Figura 17: schedule.html

- reservations: è la pagina dedicata all'utente loggato che mostra le prenotazioni effettuate e dove è possibile eliminare tali prenotazioni.

Figura 18: reservations.html

5 Test (JUnit)

I test sono stati realizzati con l'obiettivo di testare tutte le funzionalità dell'applicativo, ponendo particolare attenzione all'esecuzione corretta dei metodi essenziali. Per realizzare i test è stato utilizzato JUnit.

Si è scelto di testare i metodi contenenti la vera e propria logica di business evitando di testare metodi banali come getter e setter. La mancanza di alcune classi service è dovuta proprio al fatto che queste compiono la semplice funzione di intermediari tra il proprio controller e la rispettiva repository.

Di seguito si riporta la directory dei test e l'esito di questi ultimi.



Figura 19: Directory dei Test eseguiti

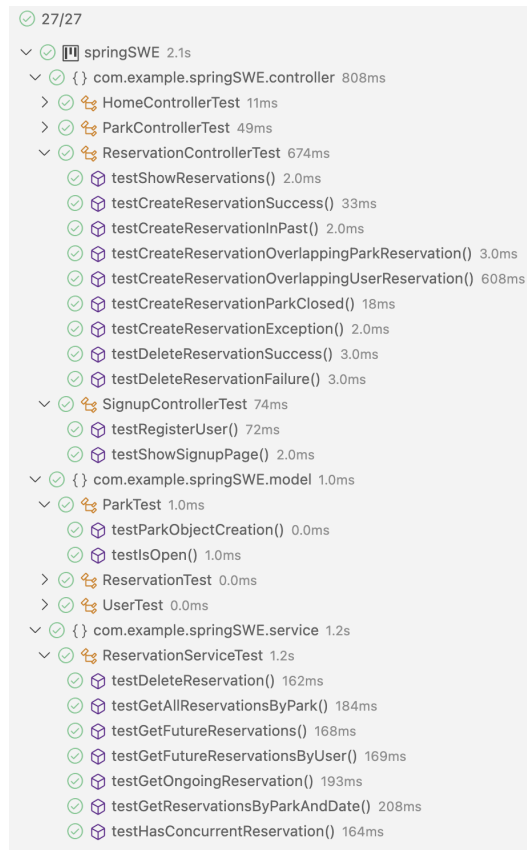


Figura 20: Esito dei test eseguiti

5.1 Model Test

In ParkTest così come in tutti i test delle entità viene testata la corretta creazione dell'oggetto verificando la coerenza degli attributi con i dati inseriti. Nel secondo test invece andiamo a testare il metodo isOpen del parco per verificarne il corretto funzionamento anche nei casi limite.

```
@Test
public void testParkObjectCreation() {
    String name = "Central Park";
    String address = "123 Park Avenue";
    LocalTime openingTime = LocalTime.of(8, 0);
    LocalTime closingTime = LocalTime.of(20, 0);

    Park park = new Park();
    park.setName(name);
    park.setAddress(address);
    park.setOpeningTime(openingTime);
    park.setClosingTime(closingTime);

    assertEquals(name, park.getName());
    assertEquals(address, park.getAddress());
    assertEquals(openingTime, park.getOpeningTime());
    assertEquals(closingTime, park.getClosingTime());
}

@Test
void testIsOpen() {
    Park park = new Park();
    park.setOpeningTime(LocalTime.of(8, 0));
    park.setClosingTime(LocalTime.of(20, 0));
    assertTrue(park.isOpen(LocalTime.of(10, 0)));
    assertTrue(park.isOpen(LocalTime.of(8, 0)));
    assertTrue(park.isOpen(LocalTime.of(20, 0)));
    assertFalse(park.isOpen(LocalTime.of(7, 59)));
    assertFalse(park.isOpen(LocalTime.of(20, 1)));
}
```

In `ReservationControllertest` viene testata la corretta istanziazione di un nuovo oggetto di tipo `Reservation` e nel secondo test viene controllato il corretto aggiornamento dell'attributo `endTime`. Questo è infatti un dato superfluo quando viene istanziato l'oggetto in quanto possiamo ricavarlo sommando all'orario di inizio la durata della prenotazione.

```
@Test
public void testReservationObjectCreation() {
    LocalDate date = LocalDate.of(2024, 12, 15);
    LocalTime startTime = LocalTime.of(10, 0);
    int duration = 20;
    User user = new User();
    Park park = new Park();

    Reservation reservation = new Reservation(date, startTime, duration, user, park);

    assertEquals(date, reservation.getDate());
    assertEquals(startTime, reservation.getStartTime());
    assertEquals(startTime.plusMinutes(duration), reservation.getEndTime());
    assertEquals(duration, reservation.getDuration());
    assertEquals(user, reservation.getUser());
    assertEquals(park, reservation.getPark());
}

@Test
public void testSetStartTimeUpdatesEndTime() {
    LocalDate date = LocalDate.of(2024, 12, 15);
    LocalTime startTime = LocalTime.of(10, 0);
    int duration = 60;

    Reservation reservation = new Reservation(date, startTime, duration, new User(), new Park());
    reservation.setStartTime(LocalTime.of(11, 0));

    assertEquals(LocalTime.of(11, 0), reservation.getStartTime());
    assertEquals(LocalTime.of(12, 0), reservation.getEndTime());
}
```

5.2 Controller Test

Passiamo adesso ai test sui controller. In `HomeControllerTest` si verifica il corretto funzionamento dei metodi del controller che gestisce l'homepage. In particolare controlliamo che alle richieste di visualizzazione della homepage venga restituita la view home e, allo stesso modo, alle richieste di login venga restituita la view login.

```
@Test
void testHomePageMapping() {
    String viewName = homeController.home();
    assertEquals("home", viewName);
}
```

Il primo test mostra come viene testato il metodo `showParks` utile nella view `parks` (Fig. 15). Il secondo test invece controlla che il metodo `getAllReservationsForPark()` ritorni correttamente tutte le prenotazioni effettuate su quel parco. In questo secondo test adoperiamo un'altra tecnologia chiamata Mockito che ci aiuta a creare dei mock di oggetti necessari in fase di test senza dover istanziare l'oggetto stesso ma semplicemente descrivendone il comportamento attraverso il metodo `when()`.

```
@Test
void testShowParks() {
    List<Park> parks = Arrays.asList(new Park(), new Park());
    when(parkService.findAllParks()).thenReturn(parks);

    String viewName = parkController.showParks(model);

    assertEquals("parks", viewName);
    verify(parkService).findAllParks();
    verify(model).addAttribute("parks", parks);
}

@Test
void testGetAllReservationsForPark() {
    Park park = new Park();
    List<Reservation> reservations = Arrays.asList(new Reservation(), new Reservation());

    // mockito when() used to mock the behavior of parkService.findParkById()
    when(parkService.findParkById(parkId)).thenReturn(park);
    when(reservationService.getFutureReservations(parkId)).thenReturn(reservations);

    String viewName = parkController.getAllReservationsForPark(parkId, model);

    assertEquals("schedule", viewName);
    // mockito verify that the findParkById method was called with the correct parkId
    verify(parkService).findParkById(parkId);
    verify(reservationService).getFutureReservations(parkId);
    verify(model).addAttribute("park", park);
    verify(model).addAttribute("reservations", reservations);
}
```

In `ReservationController` si testa le funzioni più importanti dell'applicazione, infatti ha bisogno di una struttura più complessa. Attraverso l'uso dell'annotazione `@BeforeEach` si può definire delle fixtures da utilizzare in ogni test senza dover ripetere lo stesso codice più e più volte.

```
@BeforeEach
void setUp() {
    MockitoAnnotations.openMocks(this);
    redirectAttributes = new RedirectAttributesModelMap();
    model = new ExtendedModelMap();

    // test data
    testUser = new User();
    testUser.setUsername("testUser");
    testUser.setEmail("testUser@example.com");
    testUser.setPassword("password");

    testPark = new Park();
    testPark.setName("Test Park");
    testPark.setAddress("123 Test St");

    testReservation = new Reservation(LocalDate.now().plusDays(1), LocalTime.of(12, 0),
                                     20, testUser, testPark);
}
```

`testShowReservations()` mira a testare la funzionalità che viene sfruttata nella sezione "My Reservation" (UC-2.2) in cui vengono mostrate all'utente tutte le sue prenotazioni future. Sfruttiamo ancora una volta l'uso di Mockito per descrivere come si deve comportare un mock di un oggetto `UserService` e dell'oggetto testato stesso.

```
@Test
void testShowReservations() {
    when(principal.getName()).thenReturn("testUser");
    when(userService.getUserByUsername("testUser")).thenReturn(testUser);
    List<Reservation> reservations = List.of(testReservation);
    List<Park> parks = List.of(testPark);
    when(reservationService.getFutureReservationsByUser(testUser)).thenReturn(reservations);
    when(parkService.findAllParks()).thenReturn(parks);

    String viewName = reservationController.showReservations(principal, model);

    verify(userService, times(1)).getUserByUsername("testUser");
    verify(reservationService, times(1)).getFutureReservationsByUser(testUser);
    verify(parkService, times(1)).findAllParks();
    assertEquals(reservations, model.getAttribute("reservations"));
    assertEquals(parks, model.getAttribute("parks"));
    assertEquals("reservations", viewName);
}
```

Allo stesso modo sviluppiamo tutti i test di seguito:

- **testCreateReservationSuccess** garantisce che, in uno scenario positivo (nessun errore o conflitto), il metodo salvi correttamente una nuova prenotazione, reindirizzi l'utente alla pagina delle prenotazioni e mostri un messaggio di successo.
- **testCreateReservationInPast** verifica che il metodo `createReservation` impedisca correttamente la creazione di una prenotazione nel passato, reindirizzando l'utente alla pagina precedente e mostrando un messaggio di errore ("Cannot book a reservation in the past.").
- **testCreateReservationOverlappingParkReservation** verifica che il metodo `createReservation()` impedisca correttamente la creazione di una prenotazione in caso di sovrapposizione con un'altra prenotazione per lo stesso parco, reindirizzando l'utente alla homepage e mostrando un messaggio di errore ("Selected time is already booked.").
- **testCreateReservationParkClosed()** verifica che il metodo `createReservation()` impedisca la creazione di una prenotazione quando il parco è chiuso all'orario richiesto, reindirizzando l'utente alla homepage e mostrando un messaggio di errore ("Park is closed at selected time.").
- **testCreateReservationException()** verifica che il metodo `createReservation()` gestisca correttamente un'eccezione lanciata durante la creazione di una prenotazione, reindirizzando l'utente alla homepage e mostrando un messaggio di errore contenente il motivo dell'eccezione.
- **testDeleteReservationSuccess()** verifica che il metodo `deleteReservation()` elimini correttamente una prenotazione, chiamando il servizio di eliminazione, mostrando un messaggio di successo ("Reservation successfully deleted!") e reindirizzando l'utente alla pagina delle prenotazioni.
- **testDeleteReservationFailure()** verifica che il metodo `deleteReservation` gestisca correttamente un'eccezione durante l'eliminazione di una prenotazione, mostrando un messaggio di errore che include "Could not delete reservation: Reservation not found" e reindirizzando l'utente alla pagina delle prenotazioni.

6 Note sulla connessione al Database

6.1 DBMS: PostgreSQL

Il DBMS utilizzato è PostgreSQL. È un RDBMS (DBMS relazionale) in cui i dati sono organizzati in tabelle messe in relazione tra loro.

In questo caso, le classi Java rappresenteranno le entità, con gli attributi come colonne, e gli oggetti i records.

Per praticità il server Postgres è stato installato in locale, sulla stessa macchina su cui il progetto è stato sviluppato.

Per evitare di contaminare i dati contenuti del Database dell'applicazione chiamato "app", è stato creato sullo stesso server un secondo database chiamato "test" per testare anche l'esecuzione di test su dati presenti nel database.

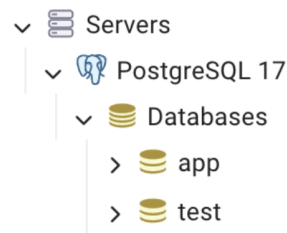


Figura 21: Database utilizzati per Applicazione e fase di Test

In questa sezione è possibile visualizzare le varie classi java istanziate come entità nel database (parks, users e reservations).

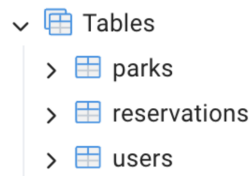


Figura 22: Tabelle presenti nel DB

Nel file application.properties, vengono scritti DBMS, indirizzo IP e porta per accedere al database.

```

1  spring.application.name=springSWE
2  spring.datasource.url=jdbc:postgresql://localhost:5433/app
3  # test database: test
4  # app database: app

```

Figura 23: Snippet di application.properties

La prima volta in cui viene inserita l'annotazione @Entity in una classe ed eseguito il programma, il DBMS procederà a effettuare automaticamente una query di creazione e inserirà come colonne tutti gli attributi.